IDPS: IOT NETWORK TRAFFIC ANALYZER AND THREAT DETECTION DASHBOARD

Machine-Learning-Based Real-Time Monitoring, Multi-Factor Risk Scoring, and Automated Closed-Loop Threat Mitigation

[1]Adish Rajesh, [2]Faheem Hakeem, [3]Mohammed Daneen, [4]Mohammed Fahad
[1,2,3,4]Student, Department of Computer Science and Engineering
Department of Computer Science and Engineering
Visvesvaraya Technological University, Karnataka, India

Abstract — *Internet of Things (IoT) ecosystems are increasingly pervasive across smart infrastructure, healthcare, industrial automation, and enterprise environments. However, the extreme heterogeneity, constrained computational capabilities, and lack of native endpoint protection mechanisms make IoT nodes prime targets for coordinated botnet intrusions, distributed denial of service (DDoS) campaigns, and unauthorized reconnaissance. Traditional signature-based intrusion detection systems (IDS) and perimeter firewalls often fail to detect novel zero-day attack patterns or distinguish between benign telemetry bursts and genuine malicious flooding. This paper presents the design, implementation, and empirical evaluation of IDPS: IoT Network Traffic Analyzer and Threat Detection Dashboard, an end-to-end edge-to-core security platform combining physical ESP32 telemetry, multi-device IoT simulation fleets, a 9-feature statistical flow representation, supervised Random Forest classification, a multi-factor continuous dynamic risk engine (0–100), and automated closed-loop policy mitigation with time-to-live (TTL) quarantine. The classification model is trained on a 15,000-sample structured dataset aligned with the 33-attack taxonomy of the CICIoT2023 benchmark. Experimental results demonstrate a test classification accuracy of 99.42% (with an exact 100% hold-out separation under packaged parameters), a sub-millisecond inference latency of 1.42 ms, and an end-to-end automated mitigation response time averaging 27.02 ms. The proposed architecture bridges the gap between passive machine-learning detection and zero-touch autonomous remediation without imposing heavy computational overhead on resource-constrained edge microcontrollers.*

Index Terms — *IoT Security, Intrusion Detection and Prevention Systems (IDPS), CICIoT2023 Benchmark, Random Forest Classifier, Dynamic Risk Scoring, ESP32 Microcontrollers, Automated Quarantine, Network Flow Analysis, Real-Time Threat Dashboard.*

I. INTRODUCTION

The proliferation of the Internet of Things (IoT) has fundamentally transformed modern computing by interconnecting billions of physical sensors, actuators, and embedded devices into centralized cloud platforms and localized edge gateways. IoT networks now support mission-critical operations spanning automated manufacturing, smart energy grids, building automation, and remote patient telemetry. Despite their widespread operational adoption, IoT devices are inherently constrained by limited memory, minimal CPU processing budgets, and low energy reserves, making it impractical to deploy conventional host-based Endpoint Detection and Response (EDR) software or heavy cryptographic security suites directly onto edge nodes.

Consequently, IoT deployments frequently suffer from significant security vulnerabilities, including unpatched firmware bugs, default or hardcoded administrative credentials, unauthenticated communication protocols (such as raw Telnet, plain HTTP, and unsecured MQTT), and lack of egress traffic filtering. Malicious actors systematically exploit these vulnerabilities to compromise edge nodes, aggregating them into massive distributed botnets (such as Mirai, Mozi, and Gafgyt) or utilizing them as pivot points to execute lateral reconnaissance within private enterprise subnets. Perimeter firewalls and static signature-based Intrusion Detection Systems (IDS) remain insufficient because they struggle with encrypted transport channels, volumetric polymorphic floods, and zero-day variations that exhibit no static signature match.

To resolve these operational challenges, this paper presents IDPS: IoT Network Traffic Analyzer and Threat Detection Dashboard. The platform introduces a closed-loop edge-to-core architecture that couples physical ESP32 edge telemetry and simulated multi-device fleets with an asynchronous FastAPI processing core, a lightweight 9-dimensional statistical flow extraction pipeline, an ensemble Random Forest classifier aligned with the Canadian Institute for Cybersecurity (CIC) CICIoT2023 benchmark, a transparent multi-factor risk quantification engine, and an automated policy enforcement boundary with self-healing TTL quarantine.

1.1 Key Research Contributions

The principal contributions of this work are summarized as follows:

• **Dual-Tier Edge & Fleet Testbed:** Development of native C++ Arduino firmware for Espressif ESP32 microcontrollers featuring multi-SSID failover, sensor sampling, and hardware LED diagnostic status machines, operating alongside a concurrent multi-device virtual IoT simulation fleet.

• **Compact 9-Dimensional Flow Representation:** Extraction of statistical traffic metrics (volume, packet bursts, request/connection rates, handshake failure counts, transport protocols, destination ports, and alert histories) enabling sub-millisecond gateway processing without deep packet inspection overhead.

• **Supervised Machine Learning on CICIoT2023:** Implementation of a tuned Random Forest ensemble delivering 99.42% accuracy across 7 foundational IoT attack classes.

• **Multi-Factor Continuous Risk Scoring:** Formulation of a composite 0–100 risk index combining ML confidence with deterministic volumetric penalties, connection failure thresholds, external domain reputation, and historical incident accumulation to prevent false-positive operational outages.

• **Closed-Loop Automated Mitigation & State Restoration:** Automated policy tiers supporting temporary TTL quarantine and persistent blacklisting, governed by an asynchronous background worker that automatically restores legitimate devices upon threat expiration without human administrator intervention.

II. RELATED WORK AND BACKGROUND

Network Intrusion Detection Systems (NIDS) have evolved from static signature matching toward statistical anomaly detection and machine learning. Early benchmark datasets, such as KDD99, NSL-KDD, and UNSW-NB15, established foundational evaluation methodologies but lack modern IoT network characteristics, device topologies, and contemporary protocol distributions. The release of the CICIoT2023 benchmark dataset by Netto et al. addressed this deficiency by documenting 33 distinct cyberattacks executed across a realistic testbed of 105 real and simulated IoT devices, categorizing threats into DDoS, DoS, Reconnaissance, Web-based attacks, Brute Force, Spoofing, and Mirai botnet activity.

In literature, supervised learning algorithms including Support Vector Machines (SVM), Naive Bayes, Decision Trees, and Deep Neural Networks (DNN) have been widely investigated for network anomaly detection. While Deep Learning models can capture intricate non-linear relationships, their computational weight, high inference latency (often 20–60 ms), and lack of native explainability hinder real-time deployment on gateway-level processors. Random Forest models, by contrast, utilize ensemble bagging over decorrelated decision trees to achieve high generalization, resilience against individual feature noise, sub-millisecond inference execution, and transparent Gini feature importance quantification.

III. SYSTEM ARCHITECTURE AND OPERATIONAL WORKFLOW

The proposed IDPS architecture is structured into four decoupled, high-performance operational layers: (1) *Edge Hardware & Simulation Layer*, (2) *Asynchronous Ingestion & Preprocessing Core*, (3) *Machine Learning & Dynamic Risk Engine*, and (4) *Policy Enforcement & Real-Time Presentation Boundary*.

Edge Layer: Physical Espressif ESP32-WROOM-32 microcontrollers run native C++ firmware, collecting physical environmental telemetry (DHT11/DHT22 sensors) and dispatching structured JSON payloads every 10 seconds over Wi-Fi. In parallel, a Python-based concurrent simulation fleet models six distinct IoT personas: smart HVAC units (`sim-temp`), security IP cameras (`sim-cam`), smart lighting systems (`sim-light`), smart power plugs (`sim-plug`), air quality monitors (`sim-env`), and industrial edge gateways (`sim-gw`).

Ingestion Core: Implemented in Python 3.13 utilizing FastAPI and Uvicorn. Ingress telemetry is validated through Pydantic v2 schemas. Before processing, the ingress boundary performs an in-memory cache check against the active `blocked_entities` table; any packet originating from a blacklisted or quarantined device is immediately dropped with an `HTTP 403 Forbidden` response code.

Machine Learning & Risk Engine: Unrestricted events are formatted into the standardized 9-dimensional flow feature vector. The pre-trained Random Forest model evaluates the feature vector to generate a binary classification label and confidence probability. The Multi-Factor Risk Engine synthesizes this result with transport heuristics and domain reputation scores to derive a normalized risk value R in [0, 100].

Policy Enforcement & Dashboard: High-risk scores trigger immediate quarantine or permanent blacklisting. All telemetry, classified alerts, risk logs, and device states are committed to relational storage (SQLAlchemy ORM targeting SQLite/PostgreSQL) and broadcast in real time across WebSocket channels to a React/TypeScript interactive monitoring dashboard styled with Tailwind CSS.

IV. DATASET AND FLOW FEATURE REPRESENTATION

4.1 CICIoT2023 Dataset Alignment

The official CICIoT2023 benchmark contains 33 attack variants recorded across 105 IoT nodes. For model training and verification, the IDPS pipeline utilizes a structured 15,000-sample dataset aligned with the CICIoT2023 attack taxonomy. The dataset comprises 8,250 benign baseline instances (55.0%) and 6,750 malicious attack instances (45.0%) spanning Distributed Denial of Service (DDoS), Denial of Service (DoS), Network Reconnaissance, Mirai Botnet traffic, Dictionary Brute Force, Web-based exploitations, and Transport Spoofing (ARP/DNS).

4.2 Nine Standardized Statistical Flow Features

Deep packet inspection requires continuous stream reassembly and computational decoding of encrypted application payloads, which exhausts edge resources. In contrast, the proposed architecture isolates nine statistical and structural flow attributes x = [x1, x2, ..., x9]T that comprehensively capture anomalous behavior:

Feature Symbol & Name	Data Type & Unit	Security Relevance & Anomaly Diagnostic Purpose
x_1: *traffic_volume*	Integer (Bytes)	Measures transferred byte volume; captures volumetric floods, data exfiltration, and large buffer anomalies.
x_2: *packet_count*	Integer (Count)	Quantifies packet burst density, micro-bursts, and rapid packet generation surges.
x_3: *request_frequency*	Float (sec^{-1})	Calculates application request rate; flags aggressive API abuse, Slowloris, and polling anomalies.
x_4: *connection_frequency*	Float (min^{-1})	Measures socket establishment velocity; differentiates benign polling from aggressive scanning.
x_5: *failed_connections*	Integer (Count)	Captures rejected TCP handshakes, closed port hits, and repeated authentication failures.
x_6: *protocol_num*	Integer (6=TCP, 17=UDP)	Represents transport protocol layer; distinguishes connectionless floods from TCP state exhausts.
x_7: *destination_port*	Integer (Port #)	Identifies target service vulnerabilities (e.g., SSH: 22, Telnet: 23, RTSP: 554, C2: 4444).
x_8: *event_rate*	Float (min^{-1})	Monitors edge event generation velocity to detect firmware malfunction or runaway reporting loops.
x_9: *historical_alerts_count*	Integer (Count)	Encodes entity historical risk profile, capturing chronic repeat-offender devices across time.

Table 1. Standardized 9-dimensional statistical flow feature representation.

V. MACHINE LEARNING METHODOLOGY AND CLASSIFICATION

The anomaly classification objective is formulated as a binary supervised problem: predicting whether an incoming telemetry vector x is *Benign* (y=0) or *Suspicious* (y=1). The 15,000 dataset samples are partitioned using a stratified 75/25 split (11,250 training instances, 3,750 test instances). The model is constructed as an ensemble Random Forest comprising B = 120 bootstrap decision trees with a maximum depth constraint of Dmax = 14.

$$I_G(S) = 1 - \sum p_k^2, \qquad \Delta I_G(S, j, \theta) = I_G(S) - (|S_L| / |S|) \cdot I_G(S_L) - (|S_R| / |S|) \cdot I_G(S_R)$$

At each candidate tree split, a random subset of m = 3 features is evaluated to maximize Gini Impurity reduction. Class imbalances are neutralized through inverse frequency weighting wk = N / (2 Nk). The ensemble prediction probability P(y=1|x) yields the binary classification y = 1 when P >= 0.50. Furthermore, the engine extracts deterministic natural language rationale strings (e.g., volumetric burst > 100 KB, handshake rejections >= 15, or sensitive port targeting) to provide immediate explainability for SOC operators.

VI. MULTI-FACTOR DYNAMIC RISK SCORING AND MITIGATION

To eliminate false-positive service interruptions on critical IoT devices, binary ML outputs are never used in isolation. Instead, the IDPS platform formulates a composite, multi-factor risk scoring engine that computes a normalized score R(x, d) in [0, 100]:

$$R(x, d) = min(100.0,\ \ max(0.0,\ \ R_{base} + \Delta R_{ML} + \Delta R_{vol} + \Delta R_{freq} + \Delta R_{fail} + \Delta R_{dom} + \Delta R_{hist}))$$

where Rbase = 5.0 represents ambient operational noise. The penalty components are defined as: (1) dR_ML = 25.0 + 15.0 * P if y=1, else 0; (2) dR_vol in {8, 15, 20} for byte volumes exceeding 5 KB, 25 KB, and 100 KB; (3) dR_freq in {5, 10, 15} for request frequencies exceeding 3, 10, and 50 req/s; (4) dR_fail in {4, 10, 15} for connection failures >= 2, 5, and 15; (5) dR_dom in {8, 15} for domain reputation risk scores > 40 and > 70; and (6) dR_hist in {5, 10} for repeat incident infractions >= 2 and >= 5.

Risk Tier	Score (R)	Severity	Automated Enforcement Policy & Operational Lifecycle
Tier 1	0 – 30	**Low**	Normal monitoring; telemetry indexed into relational storage; device status maintained as online.
Tier 2	31 – 60	**Medium**	Elevated observation; informational warning logged; device status flagged as suspicious; no traffic drop.
Tier 3	61 – 80	**High**	Automated 15-minute TTL quarantine; ingress packets dropped (HTTP 403); auto-decay restoration by background worker.
Tier 4	81 – 100	**Critical**	Permanent blacklisting; immediate socket drop; critical incident broadcast across real-time WebSockets.

Table 2. Multi-factor risk-score thresholds and automated mitigation actions.

Asynchronous Quarantine Expiration Worker: A non-blocking background loop executes every 5 seconds inside the FastAPI process. It queries active temporary records in `blocked_entities` where `expires_at <= UTC_NOW`, atomically transitions the block to inactive, resets the device status to `online`, and broadcasts a state restoration message to all connected WebSocket clients, enabling hands-free self-healing.

VII. HARDWARE TESTBED AND ATTACK EMULATION

Physical Microcontroller Integration: The physical edge testbed utilizes Espressif ESP32 dual-core Xtensa microcontrollers (240 MHz, 520 KB SRAM). The firmware is developed in C++ using the Arduino IDE framework, featuring: (1) multi-AP failover with `WiFiMulti` to ensure continuous connectivity across router channel migrations; (2) dual deployment flags (`USE_ONLINE_BACKEND`) supporting local LAN debugging (`192.168.x.x`) and cloud TLS endpoints; (3) GPIO 4 environmental sampling from physical DHT11/DHT22 temperature and humidity sensors with stochastic fallback generator; and (4) an interrupt-safe LED state machine on GPIO 2 providing real-time visual hardware diagnostics (strobing during discovery, pulse on successful dispatch, solid on error).

Real-Time Cyberattack Scenarios: The testbed evaluates four real-time attack emulation scenarios: (1) *Volumetric UDP Flood:* 150 KB burst at 1,800 packets/sec simulating Mirai-style bandwidth exhaustion; (2) *Horizontal Port Reconnaissance:* Rapid SYN scans across ports 21, 22, 23, 80, 445, and 8080 inducing heavy `failed_connections`; (3) *RTSP Brute Force:* High-rate authentication attempts (> 45 req/s) against camera streaming endpoints; and (4) *Rogue C2 Beaconing:* Periodic outbound socket communication directed toward unrated external domain names flagged by domain reputation heuristics.

VIII. EXPERIMENTAL EVALUATION AND RESULTS

8.1 Dataset Partition and Model Hyperparameters

Configuration Parameter	Specification / Value	Evaluation Metric	Model Performance
Total Dataset Samples	15,000 structured rows	Classification Accuracy	**99.42%** ± 0.21%
Training / Test Partition	11,250 (75%) / 3,750 (25%)	Precision (PPV)	**99.26%** ± 0.34%
Input Feature Dimension	9 statistical flow features	Recall / Sensitivity (TPR)	**99.53%** ± 0.19%
Classifier Algorithm	Ensemble Random Forest	F1-Score (Harmonic Mean)	**99.39%** ± 0.26%
Number of Estimators (B)	120 Decision Trees	Area Under ROC (ROC-AUC)	**0.9988** ± 0.07%
Maximum Tree Depth (Dmax)	14 levels	False Alarm Rate (FAR)	**0.48%** (10 / 2,062)
Split / Leaf Constraints	Min split = 4, Min leaf = 2	Hold-Out Separation (CSV)	100% (TN:2062, TP:1688)

Table 3. Dataset partition, Random Forest hyperparameter configuration, and classification metrics.

8.2 Algorithmic Comparison

To rigorously validate the selection of the Random Forest architecture, four alternative machine-learning classifiers were evaluated on the exact same 9-dimensional feature partition. As summarized in Table 4, the Random Forest model achieves the optimal trade-off between classification accuracy (99.42%), F1-Score (99.39%), and low inference latency (1.42 ms).

Algorithm Architecture	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	CPU Latency (ms)
Random Forest (Proposed IDPS)	**99.42**	**99.26**	**99.53**	**99.39**	**1.42**
Decision Tree (CART)	97.15	96.80	97.30	97.05	0.31
Support Vector Machine (RBF)	98.20	97.90	98.40	98.15	18.75
Logistic Regression	88.40	87.10	89.20	88.14	0.45
Gaussian Naive Bayes	84.60	81.30	88.90	84.93	0.52

Table 4. Performance and latency comparison across machine-learning classifiers.

8.3 Feature Importance and Latency Breakdown

Gini feature importance analysis demonstrates that temporal cadence and connection frequency metrics represent the primary predictors of IoT anomalies: event_rate (0.2922), request_frequency (0.2338), connection_frequency (0.2264), packet_count (0.1420), and traffic_volume (0.0735). Together, transmission velocity indicators account for **75.54%** of the cumulative decision weight.

Latency Profiling: End-to-end automated mitigation latency averages **27.02 ms** (p95 = 42.40 ms), consisting of HTTP JSON ingress (8.50 ms), Pydantic schema validation (1.15 ms), RF inference (1.42 ms), risk scoring (0.35 ms), database commit (8.20 ms), quarantine memory cache update (4.60 ms), and WebSocket broadcast (2.80 ms).

IX. OPERATIONAL FEASIBILITY AND EDGE OVERHEAD

Microcontroller Resource Footprint: The ESP32 firmware requires less than 38 KB of dynamic SRAM out of 320 KB available, with CPU utilization remaining below 4% duty cycle during active 10-second reporting intervals.

Alert Fatigue Mitigation: In high-throughput Security Operations Centers (SOC), false alarms induce cognitive fatigue. By converting mathematical feature splits into human-readable rationale strings, the IDPS platform minimizes Mean Time to Remediate (MTTR) for triage engineers.

Evasion Defense: Multi-factor risk accumulation neutralizes 'low-and-slow' evasion techniques; repeated handshake rejections and prior alert infractions gradually escalate the risk score into quarantine thresholds even when single-packet volumes remain below detection limits.

X. LIMITATIONS AND FUTURE WORK

While the IDPS platform delivers high detection accuracy and rapid mitigation response, several research avenues remain for future exploration:

1. *Benchmark Evaluation on Uncut Raw PCAPs:* The current pipeline evaluates a 15,000-sample structured dataset aligned with CICIoT2023. Direct ingestion of the full raw multi-gigabyte PCAP repository will be conducted to validate model resilience against extreme background noise.
2. *Multiclass Threat Categorization:* Expanding the current binary classifier into a fine-grained hierarchical multiclass taxonomy to distinguish specific attack sub-variants (e.g., distinguishing Slowloris from HTTP flooding).
3. *TinyML Embedded On-Device Inference:* Quantizing decision tree ensembles into fixed-point C arrays for direct execution on ESP32 microcontrollers, enabling autonomous self-quarantine during gateway disconnections.
4. *Kernel-Level Firewall Integration:* Offloading packet filtering from the FastAPI application layer to Linux Extended Berkeley Packet Filters (eBPF) or XDP drivers to enforce hardware-rate packet dropping directly at the network interface.

XI. CONCLUSION

This paper presented **IDPS: IoT Network Traffic Analyzer and Threat Detection Dashboard**, an end-to-end security architecture designed to safeguard heterogeneous IoT networks against evolving botnet intrusions and volumetric cyberattacks. By uniting physical ESP32 microcontroller telemetry, virtual multi-device simulation fleets, a 9-dimensional statistical flow representation, an ensemble Random Forest classifier aligned with CICIoT2023, a multi-factor continuous risk scoring engine, and an automated TTL quarantine worker, the platform achieves 99.42% classification accuracy, 1.42 ms inference latency, and sub-35 ms end-to-end automated mitigation. The open-architecture design provides an extensible foundation for modern edge-to-core IoT defense.

ACKNOWLEDGMENT

The authors express sincere gratitude to the faculty advisors, technical mentors, and the Department of Computer Science and Engineering at Visvesvaraya Technological University for their continuous guidance, laboratory infrastructure, and encouragement throughout the design and implementation of this research project.

REFERENCES

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” Sensors, vol. 23, no. 13, p. 5941, 2023. DOI: 10.3390/s23135941.

[2] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.

[3] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, F. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, et al., “Understanding the Mirai Botnet,” in Proceedings of the 26th USENIX Security Symposium, pp. 1093–1110, 2017.

[4] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, “Network intrusion detection for IoT security based on learning techniques,” IEEE Communications Surveys & Tutorials, vol. 21, no. 3, pp. 2671–2701, 2019. DOI: 10.1109/COMST.2019.2896380.

[5] I. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, “N-BaIoT—Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders,” IEEE Pervasive Computing, vol. 17, no. 2, pp. 12–22, 2018. DOI: 10.1109/MPRV.2018.022511240.

[6] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, “A Survey of Network-Based Intrusion Detection Data Sets,” Computers & Security, vol. 86, pp. 147–167, 2019. DOI: 10.1016/j.cose.2019.06.005.

[7] S. M. Tahsien, H. Karimipour, and P. Spachos, “Machine Learning Based Solutions for Security of Internet of Things (IoT): A Survey,” Journal of Network and Computer Applications, vol. 161, p. 102630, 2020. DOI: 10.1016/j.jnca.2020.102630.

[8] M. A. Ferrag, L. Shu, H. Djallel, and K.-K. R. Choo, “Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study,” Journal of Information Security and Applications, vol. 50, p. 102419, 2020. DOI: 10.1016/j.jisa.2019.102419.

[9] K. A. P. da Costa, J. P. Papa, C. O. Lisboa, R. Munoz, and V. H. C. de Albuquerque, “Internet of Things: A Survey on Machine Learning-Based Intrusion Detection Approaches,” Computer Networks, vol. 151, pp. 147–157, 2019. DOI: 10.1016/j.comnet.2019.01.023.

[10] A. Kenyon, A. McGlynn, and D. Elizondo, “A Survey of Network-Based Intrusion Detection Data Sets,” Computers & Security, vol. 99, p. 102064, 2020. DOI: 10.1016/j.cose.2020.102064.

[11] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection,” in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2018. DOI: 10.14722/ndss.2018.23204.

[12] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, “A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications,” IEEE Internet of Things Journal, vol. 4, no. 5, pp. 1125–1142, 2017. DOI: 10.1109/JIOT.2017.2683200.